```
Start coding or generate with AI.
```

```
!pip install spacy
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/pyt
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/pyt
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/pytho
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.1
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/loca
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.1
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/pytho
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/pyth
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pack
```

```
import subprocess

# Download the 'en_core_web_sm' English language model
try:
    subprocess.run(["python", "-m", "spacy", "download", "en_core_web_sm"], che
```

```
    print("Successfully downloaded 'en_core_web_sm' model.")
except subprocess.CalledProcessError as e:
    print(f"Error downloading 'en_core_web_sm' model: {e}")
except FileNotFoundError:
    print("Error: 'python' command not found. Ensure Python is in your PATH or
```

```
Successfully downloaded 'en_core_web_sm' model.
```

```
import spacy

# Load the 'en_core_web_sm' English language model
try:
    nlp = spacy.load("en_core_web_sm")
    print("Successfully loaded 'en_core_web_sm' model.")
except OSError:
    print("Error: 'en_core_web_sm' model not found. Please ensure it is downloa
```

```
Successfully loaded 'en_core_web_sm' model.
```

```
sentences = [
    "The quick brown fox jumps over the lazy dog.", # Simple sentence
    "She sings beautifully, and he plays the guitar skillfully.", # Compound se
    "Although it was raining, they decided to go for a walk in the park.", # Co
    "What is your name?", # Interrogative sentence
    "The ball was kicked by the boy.", # Passive voice
    "I enjoy reading books that challenge my perspective.", # Complex sentence
    "He studied hard for the exam; therefore, he passed with flying colors.", #
    "After finishing her work, Sarah went to the gym.", # Simple sentence with
    "Do you know where the nearest coffee shop is?", # Interrogative sentence,
    "The ancient ruins were discovered by a team of archaeologists last year.",
    "We went to the market, bought some fresh vegetables, and cooked a deliciou
    "The cat, a fluffy Persian, slept soundly on the couch.", # Simple sentence
    "If you work diligently, you will achieve your goals.", # Complex sentence
    "Who wrote 'To Kill a Mockingbird'?", # Interrogative sentence
    "Dinner is being prepared by my mother right now.", # Passive voice
    "The sun shines brightly.", # Simple sentence
    "I went to the store, but I forgot my wallet."
]

print(f"Created a list of {len(sentences)} sentences.")
```

```
Created a list of 17 sentences.
```

```
print("\n--- Processing Sentences with spaCy ---")
for i, sentence in enumerate(sentences):
    print(f"\nSentence {i+1}: {sentence}")
    doc = nlp(sentence)
    print("Token | POS | Dependency")
```

```
    print("-------------------------")
    for token in doc:
        print(f"{token.text} | {token.pos_} | {token.dep_}")
    # For brevity, only process and print details for the first 3 sente
    # and then show an example for the rest
    if i >= 2 and i < len(sentences) - 1: # Only show full details for
        print("...")
        break # Exit after the first few to avoid excessively long outp
```

```
--- Processing Sentences with spaCy ---

Sentence 1: The quick brown fox jumps over the lazy dog.
Token | POS | Dependency
-------------------------
The | DET | det
quick | ADJ | amod
brown | ADJ | amod
fox | NOUN | nsubj
jumps | VERB | ROOT
over | ADP | prep
the | DET | det
lazy | ADJ | amod
dog | NOUN | pobj
. | PUNCT | punct

Sentence 2: She sings beautifully, and he plays the guitar skillfully.
Token | POS | Dependency
-------------------------
She | PRON | nsubj
sings | VERB | ROOT
beautifully | ADV | advmod
, | PUNCT | punct
and | CCONJ | cc
he | PRON | nsubj
plays | VERB | conj
the | DET | det
guitar | NOUN | dobj
skillfully | ADV | advmod
. | PUNCT | punct

Sentence 3: Although it was raining, they decided to go for a walk in the park.
Token | POS | Dependency
-------------------------
Although | SCONJ | mark
it | PRON | nsubj
was | AUX | aux
raining | VERB | advcl
, | PUNCT | punct
they | PRON | nsubj
decided | VERB | ROOT
to | PART | aux
go | VERB | xcomp
for | ADP | prep
a | DET | det
```

```
walk | NOUN | pobj
in | ADP | prep
the | DET | det
park | NOUN | pobj
. | PUNCT | punct
...
```

```
print("\n--- Extracting Detailed Dependency Relations ---")
for i, sentence in enumerate(sentences):
    doc = nlp(sentence)
    print(f"\nSentence {i+1}: {sentence}")
    print("Token        | Lemma        | POS Tag      | Dependency   | Head Wor
    print("-------------------------------------------------------------------"
    for token in doc:
        print(f"{token.text:<12} | {token.lemma_:<12} | {token.pos_:<12} | {tok

    # Limit full output to the first 3 sentences for brevity, then show an exam
    if i >= 2 and i < len(sentences) - 1:
        print("... (full details for remaining sentences omitted for brevity)")
        break
```

```
--- Extracting Detailed Dependency Relations ---

Sentence 1: The quick brown fox jumps over the lazy dog.
Token        | Lemma        | POS Tag      | Dependency   | Head Word
-------------------------------------------------------------------
The          | the          | DET          | det          | fox
quick        | quick        | ADJ          | amod         | fox
brown        | brown        | ADJ          | amod         | fox
fox          | fox          | NOUN         | nsubj        | jumps
jumps        | jump         | VERB         | ROOT         | jumps
over         | over         | ADP          | prep         | jumps
the          | the          | DET          | det          | dog
lazy         | lazy         | ADJ          | amod         | dog
dog          | dog          | NOUN         | pobj         | over
.            | .            | PUNCT        | punct        | jumps

Sentence 2: She sings beautifully, and he plays the guitar skillfully.
Token        | Lemma        | POS Tag      | Dependency   | Head Word
-------------------------------------------------------------------
She          | she          | PRON         | nsubj        | sings
sings        | sing         | VERB         | ROOT         | sings
beautifully  | beautifully  | ADV          | advmod       | sings
,            | ,            | PUNCT        | punct        | sings
and          | and          | CCONJ        | cc           | sings
he           | he           | PRON         | nsubj        | plays
plays        | play         | VERB         | conj         | sings
the          | the          | DET          | det          | guitar
guitar       | guitar       | NOUN         | dobj         | plays
skillfully   | skillfully   | ADV          | advmod       | plays
.            | .            | PUNCT        | punct        | plays

Sentence 3: Although it was raining, they decided to go for a walk in the park.
```

```
Token        | Lemma        | POS Tag      | Dependency   | Head Word
--------------------------------------------------------------------
Although     | although     | SCONJ        | mark         | raining
it           | it           | PRON         | nsubj        | raining
was          | be           | AUX          | aux          | raining
raining      | rain         | VERB         | advcl        | decided
,            | ,            | PUNCT        | punct        | decided
they         | they         | PRON         | nsubj        | decided
decided      | decide       | VERB         | ROOT         | decided
to           | to           | PART         | aux          | go
go           | go           | VERB         | xcomp        | decided
for          | for          | ADP          | prep         | go
a            | a            | DET          | det          | walk
walk         | walk         | NOUN         | pobj         | for
in           | in           | ADP          | prep         | walk
the          | the          | DET          | det          | park
park         | park         | NOUN         | pobj         | in
.            | .            | PUNCT        | punct        | decided
... (full details for remaining sentences omitted for brevity)
```

```
print("--- Selecting Sentences for Detailed Analysis ---")

# Select representative sentences from the existing list
# 1. Simple sentence
# 2. Compound sentence
# 3. Complex sentence
# 4. Passive voice sentence

selected_sentences = [
    sentences[0],  # "The quick brown fox jumps over the lazy dog." (Simple)
    sentences[1],  # "She sings beautifully, and he plays the guitar skillfully
    sentences[2],  # "Although it was raining, they decided to go for a walk ir
    sentences[4]   # "The ball was kicked by the boy." (Passive Voice)
]

print(f"Selected {len(selected_sentences)} sentences for detailed analysis:")
for i, s in enumerate(selected_sentences):
    print(f"{i+1}. {s}")
```

```
--- Selecting Sentences for Detailed Analysis ---
Selected 4 sentences for detailed analysis:
1. The quick brown fox jumps over the lazy dog.
2. She sings beautifully, and he plays the guitar skillfully.
3. Although it was raining, they decided to go for a walk in the park.
4. The ball was kicked by the boy.
```

```
print("\n--- Identifying Sentence Components using Dependency Relations ---")

for i, sentence_text in enumerate(selected_sentences):
    doc = nlp(sentence_text)
    print(f"\nSentence {i+1}: {sentence_text}")
```

```
    subjects = [token.text for token in doc if "subj" in token.dep_]
    main_verbs = [token.text for token in doc if token.dep_ == "ROOT" or token.
    direct_objects = [token.text for token in doc if token.dep_ == "dobj"]
    adjectival_modifiers = [token.text for token in doc if token.dep_ == "amod"
    adverbial_modifiers = [token.text for token in doc if token.dep_ == "advmoc
    prepositional_phrases = []

    # Collect prepositional phrases by identifying 'prep' dependencies and thei
    for token in doc:
        if token.dep_ == "prep":
            prep_obj = [child.text for child in token.children if child.dep_ ==
            if prep_obj:
                prepositional_phrases.append(f"{token.text} {" ".join(prep_obj)

    print(f"  Subjects: {', '.join(subjects) if subjects else 'None'}")
    print(f"  Main Verbs: {', '.join(main_verbs) if main_verbs else 'None'}")
    print(f"  Direct Objects: {', '.join(direct_objects) if direct_objects else
    print(f"  Adjectival Modifiers: {', '.join(adjectival_modifiers) if adjecti
    print(f"  Adverbial Modifiers: {', '.join(adverbial_modifiers) if adverbial
    print(f"  Prepositional Phrases: {'; '.join(prepositional_phrases) if prepc
```

```
--- Identifying Sentence Components using Dependency Relations ---

Sentence 1: The quick brown fox jumps over the lazy dog.
  Subjects: fox
  Main Verbs: jumps
  Direct Objects: None
  Adjectival Modifiers: quick, brown, lazy
  Adverbial Modifiers: None
  Prepositional Phrases: over dog

Sentence 2: She sings beautifully, and he plays the guitar skillfully.
  Subjects: She, he
  Main Verbs: sings
  Direct Objects: guitar
  Adjectival Modifiers: None
  Adverbial Modifiers: beautifully, skillfully
  Prepositional Phrases: None

Sentence 3: Although it was raining, they decided to go for a walk in the park.
  Subjects: it, they
  Main Verbs: decided
  Direct Objects: None
  Adjectival Modifiers: None
  Adverbial Modifiers: None
  Prepositional Phrases: for walk; in park

Sentence 4: The ball was kicked by the boy.
  Subjects: ball
  Main Verbs: kicked
  Direct Objects: None
  Adjectival Modifiers: None
```

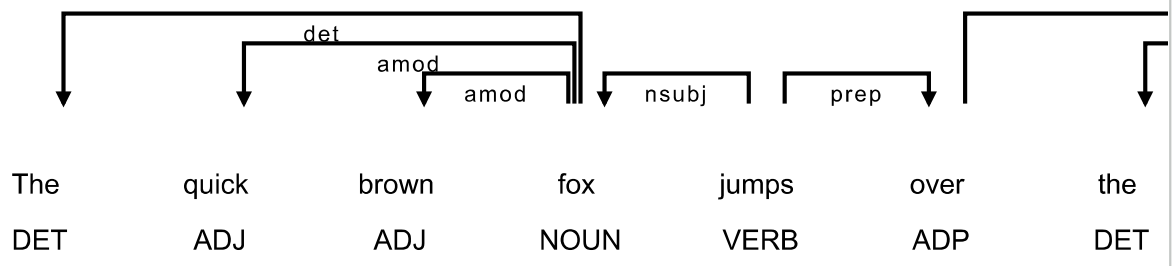
```
  Adverbial Modifiers: None
  Prepositional Phrases: None
```

```
import spacy
from spacy import displacy

# Assuming 'nlp' model is already loaded from previous steps
# If not, ensure it's loaded: nlp = spacy.load("en_core_web_sm")

print("--- Selecting Sentences for Dependency Tree Visualization ---")

# Select at least 5 representative sentences from the existing 'sentences' list
sentences_to_visualize = [
    sentences[0],  # "The quick brown fox jumps over the lazy dog." (Simple)
    sentences[1],  # "She sings beautifully, and he plays the guitar skillfully
    sentences[2],  # "Although it was raining, they decided to go for a walk ir
    sentences[4],  # "The ball was kicked by the boy." (Passive Voice)
    sentences[6]   # "He studied hard for the exam; therefore, he passed with f
]

print(f"Selected {len(sentences_to_visualize)} sentences for dependency tree vi
for i, s in enumerate(sentences_to_visualize):
    print(f"  {i+1}. {s}")
```

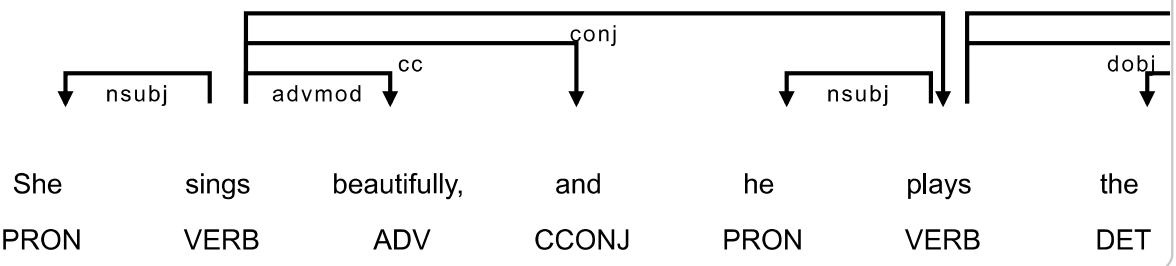
```
--- Selecting Sentences for Dependency Tree Visualization ---
Selected 5 sentences for dependency tree visualization:
  1. The quick brown fox jumps over the lazy dog.
  2. She sings beautifully, and he plays the guitar skillfully.
  3. Although it was raining, they decided to go for a walk in the park.
  4. The ball was kicked by the boy.
  5. He studied hard for the exam; therefore, he passed with flying colors.
```

```
print("\n--- Displaying Dependency Trees with displaCy ---")

for i, sentence_text in enumerate(sentences_to_visualize):
    doc = nlp(sentence_text)
    print(f"\nDependency Tree for Sentence {i+1}: {sentence_text}")
    displacy.render(doc, style="dep", jupyter=True, options={'compact': True, '
```

```
--- Displaying Dependency Trees with displaCy ---

Dependency Tree for Sentence 1: The quick brown fox jumps over the lazy dog.
```




```
Dependency Tree for Sentence 2: She sings beautifully, and he plays the guitar s
```




```
print("--- Selecting Sentences for Detailed Comparative Analysis ---")

# Re-using the sentences selected for visualization, which cover the required c
# 1. Simple sentence: "The quick brown fox jumps over the lazy dog."
# 2. Compound sentence: "She sings beautifully, and he plays the guitar skillfu
# 3. Complex sentence: "Although it was raining, they decided to go for a walk
# 4. Passive voice sentence: "The ball was kicked by the boy."
# 5. Sentence with significant modifiers (e.g., adjectival and adverbial): Can

# Let's use the same list as 'sentences_to_visualize' for consistency and to cc
analysis_sentences = sentences_to_visualize

print(f"Selected {len(analysis_sentences)} sentences for detailed comparative a
for i, s in enumerate(analysis_sentences):
    print(f"{i+1}. {s}")
```

```
--- Selecting Sentences for Detailed Comparative Analysis ---
Selected 5 sentences for detailed comparative analysis:
```

```
1. The quick brown fox jumps over the lazy dog.
2. She sings beautifully, and he plays the guitar skillfully.
3. Although it was raining, they decided to go for a walk in the park.
4. The ball was kicked by the boy.
5. He studied hard for the exam; therefore, he passed with flying colors.
```

Dependency Tree for Sentence 4: The ball was kicked by the boy.

```
print("\n--- Demonstrating Dependency Relations for Comparative Analysis ---")

for i, sentence_text in enumerate(analysis_sentences):
    doc = nlp(sentence_text)
    print(f"\nSentence {i+1}: {sentence_text}")
    print("Token        | POS Tag      | Dependency   | Head Word")
    print("-----------------------------------------------------")

    for token in doc:
        # Highlight specific dependencies for analysis
        if token.dep_ in ["ROOT", "nsubj", "dobj", "pobj", "amod", "advmod", "p
            print(f"{token.text:<12} | {token.pos_:<12} | {token.dep_:<12} | {t
        # For other tokens, just print basic info if needed, or skip for brevit
        # else:
        #     print(f"{token.text:<12} | {token.pos_:<12} | {token.dep_:<12} |

    # Additionally, print all subjects, verbs, objects, and modifiers found for
    subjects = [token.text for token in doc if "subj" in token.dep_]
    main_verbs = [token.text for token in doc if token.dep_ == "ROOT" or (token
    direct_objects = [token.text for token in doc if token.dep_ == "dobj"]
    prepositional_objects = [token.text for token in doc if token.dep_ == "pobj
    adjectival_modifiers = [token.text for token in doc if token.dep_ == "amod"
    adverbial_modifiers = [token.text for token in doc if token.dep_ == "advmod
    agents_passive = [child.text for token in doc if token.dep_ == "prep" and t

    print(f"Summary for Sentence {i+1}:")
    print(f"  Subjects: {', '.join(subjects) if subjects else 'None'}")
    print(f"  Main/Conjoined Verbs: {', '.join(main_verbs) if main_verbs else '
    print(f"  Direct Objects: {', '.join(direct_objects) if direct_objects else
    print(f"  Objects of Prepositions: {', '.join(prepositional_objects) if pre
    print(f"  Adjectival Modifiers: {', '.join(adjectival_modifiers) if adjecti
    print(f"  Adverbial Modifiers: {', '.join(adverbial_modifiers) if adverbial
    print(f"  Passive Agents (by clause): {', '.join(agents_passive) if agents_
```

```
--- Demonstrating Dependency Relations for Comparative Analysis ---

Sentence 1: The quick brown fox jumps over the lazy dog.
Token        | POS Tag      | Dependency   | Head Word
-----------------------------------------------------
quick        | ADJ          | amod         | fox
brown        | ADJ          | amod         | fox
fox          | NOUN         | nsubj        | jumps
jumps        | VERB         | ROOT         | jumps
over         | ADP          | prep         | jumps
```

```
lazy          | ADJ          | amod         | dog
dog           | NOUN         | pobj         | over
Summary for Sentence 1:
  Subjects: fox
  Main/Conjoined Verbs: jumps
  Direct Objects: None
  Objects of Prepositions: dog
  Adjectival Modifiers: quick, brown, lazy
  Adverbial Modifiers: None
  Passive Agents (by clause): None

Sentence 2: She sings beautifully, and he plays the guitar skillfully.
Token         | POS Tag      | Dependency   | Head Word
--------------------------------------------------
She           | PRON         | nsubj        | sings
sings         | VERB         | ROOT         | sings
beautifully   | ADV          | advmod       | sings
he            | PRON         | nsubj        | plays
plays         | VERB         | conj         | sings
guitar        | NOUN         | dobj         | plays
skillfully    | ADV          | advmod       | plays
Summary for Sentence 2:
  Subjects: She, he
  Main/Conjoined Verbs: sings, plays
  Direct Objects: guitar
  Objects of Prepositions: None
  Adjectival Modifiers: None
  Adverbial Modifiers: beautifully, skillfully
  Passive Agents (by clause): None

Sentence 3: Although it was raining, they decided to go for a walk in the park
Token         | POS Tag      | Dependency   | Head Word
--------------------------------------------------
Although      | SCONJ        | mark         | raining
it            | PRON         | nsubj        | raining
raining       | VERB         | advcl        | decided
they          | PRON         | nsubj        | decided
decided       | VERB         | ROOT         | decided
go            | VERB         | xcomp        | decided
for           | ADP          | prep         | go
walk          | NOUN         | pobj         | for
in            | ADP          | prep         | walk
park          | NOUN         | pobj         | in
Summary for Sentence 3:
  Subjects: it, they
  Main/Conjoined Verbs: decided
```